

Шпаргалка по коду: Реализация алгоритмов Machine Learning

23 июня 2026 г.

Основные этапы решения задачи машинного обучения:

1. Постановка задачи;
2. Выделение признаков;
3. Формирование выборки;
4. Выбор функционала ошибки;
5. Предобработка данных;
6. Построение модели;
7. Оценивание качества модели.

Содержание

1	Шпаргалка по визуализации (Matplotlib & Seaborn)	3
1.1	Эталонное оформление графиков	3
1.2	Каталог видов графиков	4
1.2.1	Сравнение величин и временные ряды	4
1.2.2	Матрицы корреляций признаков	4
1.2.3	Диаграммы рассеяния и тренды (Scatter plots)	5
1.2.4	Распределения случайных величин (Одномерные и Двумерные)	6
1.3	Справочник параметров основных функций Seaborn	7
1.3.1	Общие параметры для большинства графиков (scatterplot, histplot, lineplot, barplot)	7
1.3.2	Специфичные параметры для конкретных графиков	7
2	Matplotlib & Seaborn: Глобальные настройки и Трюки	8
2.1	Оптимизация качества и параметров по умолчанию	8
2.2	Разбор базовых настроек полотна	9
3	Библиотека Pandas: Ввод-Вывод данных (I/O)	10
3.1	Полное семейство функций чтения (read_*)	10
3.2	Полное семейство функций экспорта (to_*)	10
4	Pandas: Серии, Структура и Продвинутой Индексация	11
4.1	Работа с одномерными массивами (Series)	11
4.2	Создание и базовое исследование DataFrame	11
4.3	Различия loc и iloc при работе со срезами [:]	11
4.4	Продвинутой фильтрация по условиям (Булевы маски)	11
5	Pandas: Трансформация, Группировки и Объединения	12
5.1	Применение функций: map, apply и applymap	12
5.2	Агрегация и Группировка: groupby и agg	12
5.3	Работа с пропусками: fillna и dropna	12
5.4	Слияние и Склеивка таблиц: concat, merge, join	13
5.5	Применение функций: map, apply и applymap	13
5.6	Агрегация, Математика и Группировка: groupby и agg	13
5.7	Исследование матриц, статистики и удаление: drop и describe	14
5.8	Корреляции и матрицы сопряженности: corr, cov	14
6	Сравнение библиотек: Pandas vs Polars	15
6.1	В чем Polars лучше Pandas (Плюсы Polars)	15
6.2	В чем Polars хуже Pandas (Минусы Polars)	15
6.3	Ключевые фишки и синтаксические различия	15

7 Библиотека Polars: Ввод-Вывод данных (I/O)	16
7.1 Полное семейство функций чтения (<code>read_*</code> и <code>scan_*</code>)	16
7.2 Полное семейство функций экспорта (<code>write_*</code>)	16
8 Polars: Структура данных и Способы Индексации	17
8.1 Работа с одномерными массивами (Series)	17
8.2 Создание и базовое исследование DataFrame	17
8.3 Выборка данных и срезы (Вместо <code>loc</code> и <code>iloc</code>)	17
8.4 Продвинутое фильтрация по условиям	17
9 Polars: Трансформация, Группировки, Математика и Объединения	18
9.1 Применение выражений и кастомных функций: <code>map_elements</code>	18
9.2 Агрегация, Математика и Группировка: <code>group_by</code> и <code>agg</code>	18
9.3 Исследование матриц, статистики и удаление: <code>drop</code> и <code>describe</code>	18
9.4 Расчет матриц корреляции и ковариации	18
9.5 Работа с пропусками: <code>fill_null</code> и <code>drop_null</code>	19
9.6 Слияние и Склеивка таблиц: <code>concat</code> и <code>join</code>	19
10 NumPy для Machine Learning: Векторизация и Матрицы	20
10.1 Создание массивов и генерация данных	20
10.2 Генерация случайных величин и воспроизводимость	20
10.3 Манипуляции с формами матриц (Reshaping и Склеивание)	20
10.4 Векторизация, Агрегаты и Матричные операции	20
11 Линейная регрессия и библиотека sklearn	22
11.1 1. Теоретическая база и Разведочный анализ (EDA)	22
11.2 2. Продвинутое обработка признаков (Feature Engineering)	22
11.3 3. Преобразование целевой переменной	22
11.4 4. Сквозные Пайплайны (ColumnTransformer + Pipeline)	23
11.5 5. Регуляризация (Ridge и Lasso)	23
11.6 6. Кросс-валидация и подбор параметров (GridSearchCV)	23
12 Градиентный спуск и методы оптимизации	25
12.1 Зачем реализовывать градиентный спуск вручную, если есть <code>sklearn</code> ?	25
12.2 1. Теоретический базис и концепция антиградиента	25
12.3 2. Классический градиентный спуск (Full Gradient Descent)	25
12.4 3. Стохастический градиентный спуск (Stochastic Gradient Descent, SGD)	25
12.5 4. Динамика и регулирование темпа обучения (Learning Rate)	26
12.6 5. Промышленный SGD-оптимизатор в <code>sklearn</code> : <code>SGDRegressor</code>	26
13 Предобработка данных и Feature Engineering (Полный курс)	28
13.1 1. Пропущенные значения (Missing Values)	28
13.2 2. Категориальные признаки: OHE против Target Encoding	28
13.2.1 One-Hot Encoding (Унитарное кодирование)	28
13.2.2 Target Encoding (Кодирование целевой переменной / Счетчики)	28
13.3 3. Текстовые признаки: Частотные матрицы и TF-IDF	29
13.3.1 Bag of Words (Мешок слов) и n-граммы	29
13.3.2 TF-IDF векторизация	29
13.4 4. Лемматизация и стемминг	29
14 Предсказание и калибровка вероятностей	31
14.1 Прогнозирование вероятностей (Мягкая классификация)	31
14.2 Метрики качества вероятностей	31
14.3 Методы калибровки вероятностей	31
14.4 Мультиклассовая калибровка	32
15 Квантильная регрессия	33
15.1 Постановка задачи и функционал ошибки	33
15.2 Вероятностный смысл функционала	33
15.3 Сводная таблица аппроксимации распределений	33

1 Шпаргалка по визуализации (Matplotlib & Seaborn)

1.1 Эталонное оформление графиков

Суть: График должен содержать название, подписанные оси и легенду/шкалу.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from sklearn.datasets import make_moons
5
6 # 1. Генерация синтетических данных
7 X, y = make_moons(n_samples=500, noise=0.1, random_state=0)
8
9 # 2. Инициализация полотна на несколько графиков
10 # figsize=(ширина, высота), nrow/ncol - сетка графиков
11 # gridspec_kw - задает пропорции ширины подграфиков
12 fig, ax = plt.subplots(figsize=(16, 7), nrow=1, ncol=2, gridspec_kw={'width_ratios':
13     [1.75, 2]})
14
15 # --- Левый график: Бинарная классификация ---
16 sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=y, ax=ax[0])
17 ax[0].legend(title='class') # Настройка заголовка легенды
18 ax[0].set_title('Classification') # Заголовок подграфика
19
20 # --- Правый график: Регрессия ---
21 X_reg = np.random.randn(500, 2)
22 y_reg = np.sqrt((X_reg ** 2).sum(axis=1))
23 # palette - цветовая схема (например, 'viridis')
24 sns.scatterplot(x=X_reg[:, 0], y=X_reg[:, 1], hue=y_reg, ax=ax[1], palette='viridis', legend
25     =False)
26
27 # Настройка непрерывной цветовой шкалы (Colorbar) для регрессии
28 norm = plt.Normalize(y_reg.min(), y_reg.max())
29 sm = plt.cm.ScalarMappable(cmap='viridis', norm=norm)
30 sm.set_array([])
31 fig.colorbar(sm, ax=ax[1], label='target')
32 ax[1].set_title('Regression')
33
34 # --- Общие настройки для всех осей ---
35 for ax in ax:
36     ax.set_xlabel('$x_1$') # Подпись оси X (поддерживает LaTeX)
37     ax.set_ylabel('$x_2$') # Подпись оси Y
38
39 plt.show() # Вывод графика на экран
```

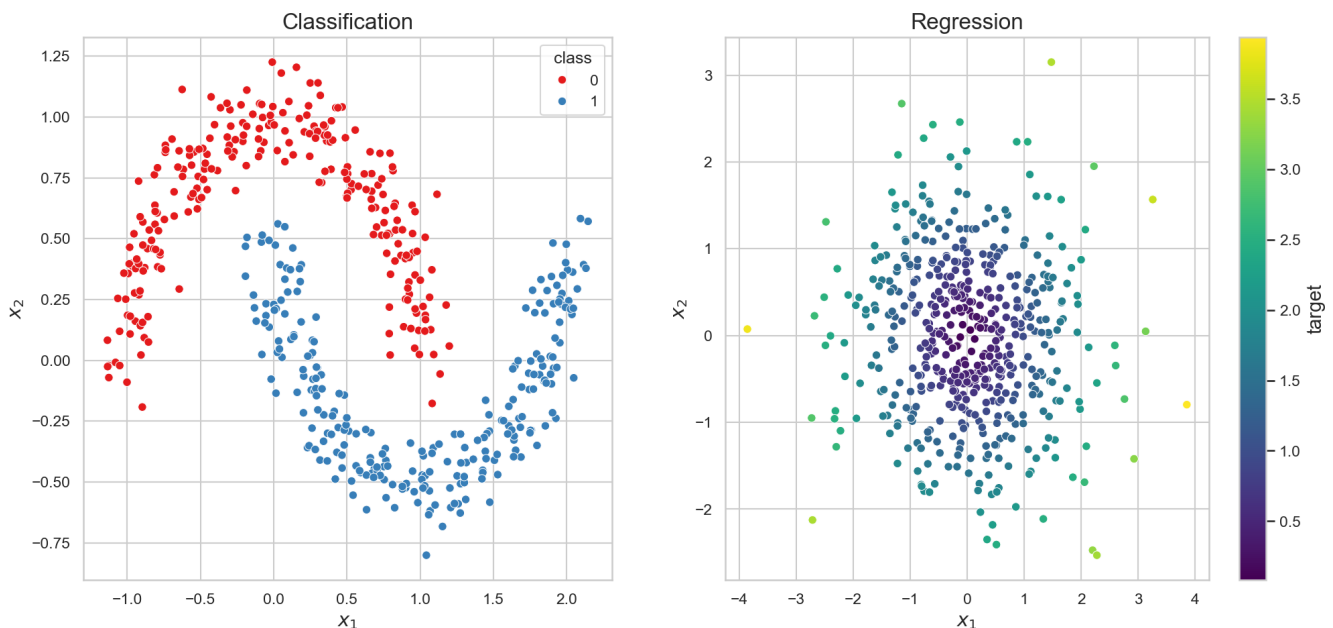


Рис. 1: Эталон разделения на подграфики (Classification и Regression)

1.2 Каталог видов графиков

1.2.1 Сравнение величин и временные ряды

`sns.barplot()` и `sns.lineplot()`

- `sns.barplot()` — Сравнение дискретных категорий. Удобно для рейтингов и средних значений по группам.
- `sns.lineplot()` — Отображение непрерывных процессов, трендов и временных рядов во времени.

```
1 # Столбчатая диаграмма по категориям
2 sns.barplot(x=ign_data['Racing'], y=ign_data.index)
3
4 # Линейный график непрерывных данных
5 sns.lineplot(data=spotify_data)
```

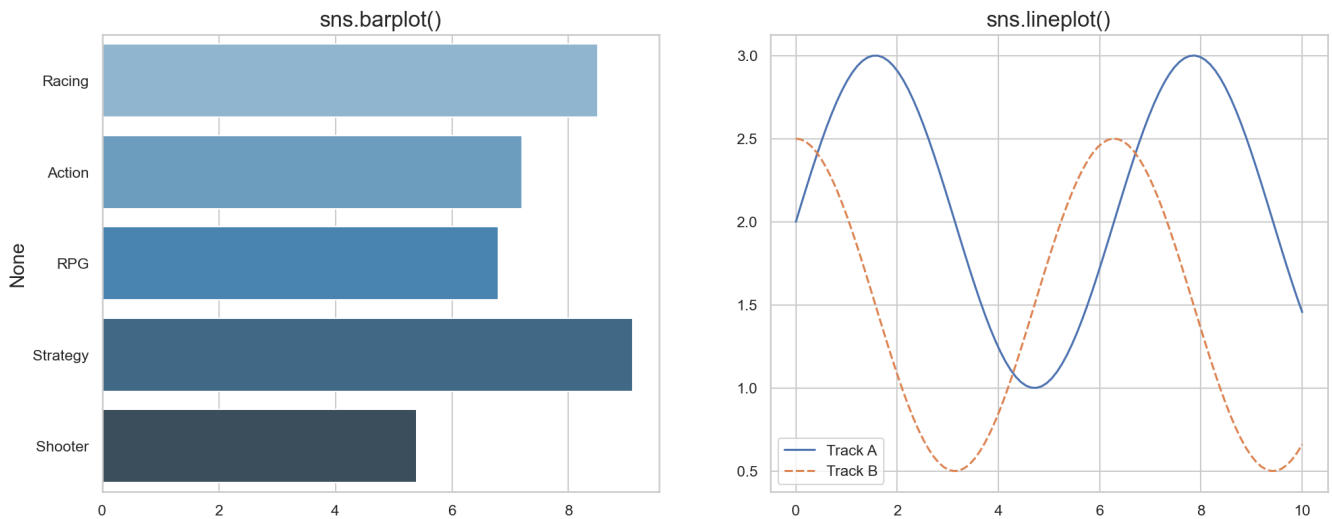


Рис. 2: Примеры категориального столбчатого и непрерывного линейного графиков

1.2.2 Матрицы корреляций признаков

`sns.heatmap()` — Визуализация матрицы парных корреляций.

- **Параметры:** `annot=True` выводит точные значения коэффициентов внутри ячеек.
- **Плюсы:** Быстрый поиск мультиколлинеарности (сильной линейной связи) между фичами.

```
1 sns.heatmap(ign_data, annot=True)
```

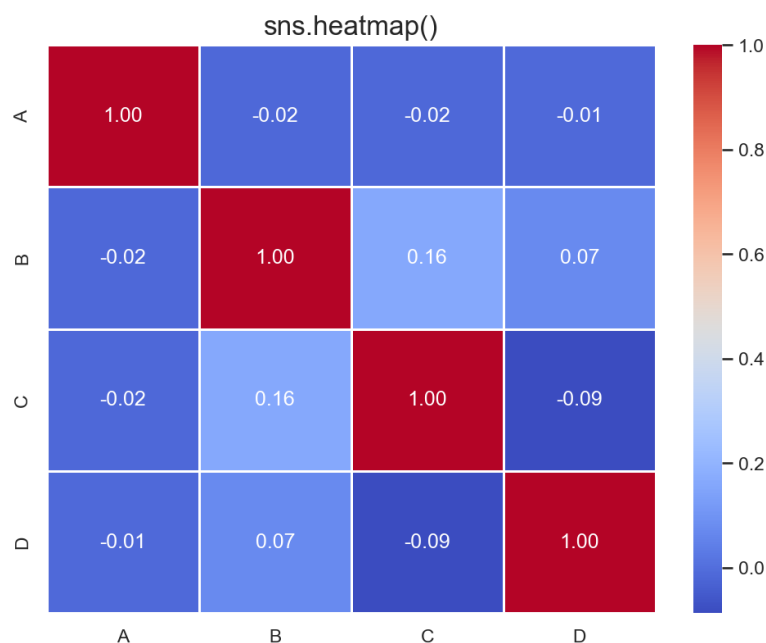


Рис. 3: Тепловая матрица корреляции признаков

1.2.3 Диаграммы рассеяния и тренды (Scatter plots)

`sns.scatterplot()` / `sns.regplot()` / `sns.swarmplot()`

- **scatterplot** — Показывает распределение точек на плоскости. Помогает увидеть кластеры и форму зависимости.
- **regplot** — Строит scatter-плот и сразу накладывает линию линейной регрессии.
- **swarmplot** — Разделяет точки по категориям так, чтобы они не накладывались друг на друга. Показывает точную плотность «облака». На больших данных жестко тормозит.

```
1 # Простой scatter и с группировкой по цветам (hue)
2 sns.scatterplot(x=candy_data['sugarpercent'], y=candy_data['winpercent'])
3 sns.scatterplot(x=insurance_data['bmi'], y=insurance_data['charges'], hue=insurance_data['smoker'])
4
5 # Построение линии регрессии тренда
6 sns.regplot(x=insurance_data['bmi'], y=insurance_data['charges'])
7
8 # Пчелиная диаграмма для категорий
9 sns.swarmplot(x=insurance_data['smoker'], y=insurance_data['charges'])
```

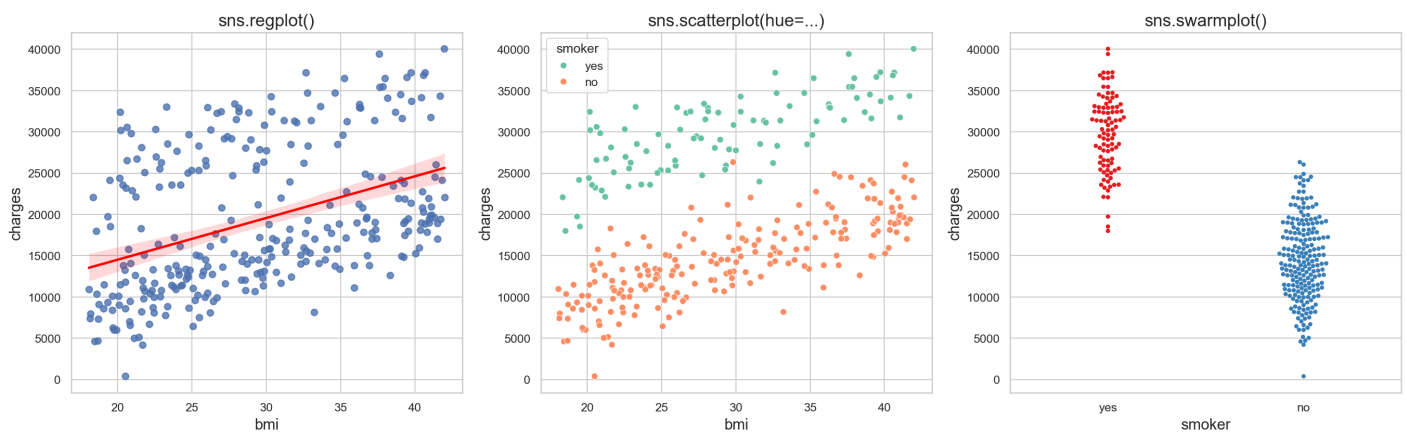


Рис. 4: Анализ зависимостей: Линейный тренд, разделение по цветам (hue) и плотность распределения категорий

1.2.4 Распределения случайных величин (Одномерные и Двумерные)

`sns.histplot()` / `sns.kdeplot()` / `sns.jointplot()`

- **histplot** — Классическая гистограмма для оценки распределения признака.
- **kdeplot** — Плавный непрерывный график оценки плотности вероятности (альтернатива ступенчатым столбцам).
- **jointplot** — Комбинирует двумерное распределение (по центру) и одномерные графики плотности (сверху и справа).

```
1 # Гистограммы (обычная и с разделением по классам)
2 sns.histplot(iris_data['Petal Length (cm)'])
3 sns.histplot(data=iris_data, x='Petal Length (cm)', hue='Species')
4
5 # Графики плотности KDE
6 sns.kdeplot(data=iris_data['Petal Length (cm)'], fill=True)
7 sns.kdeplot(data=iris_data, x='Petal Length (cm)', hue='Species', fill=True)
8
9 # Совмещенное двумерное распределение признаков
10 sns.jointplot(x=iris_data['Petal Length (cm)'], y=iris_data['Sepal Width (cm)', kind="kde")
```

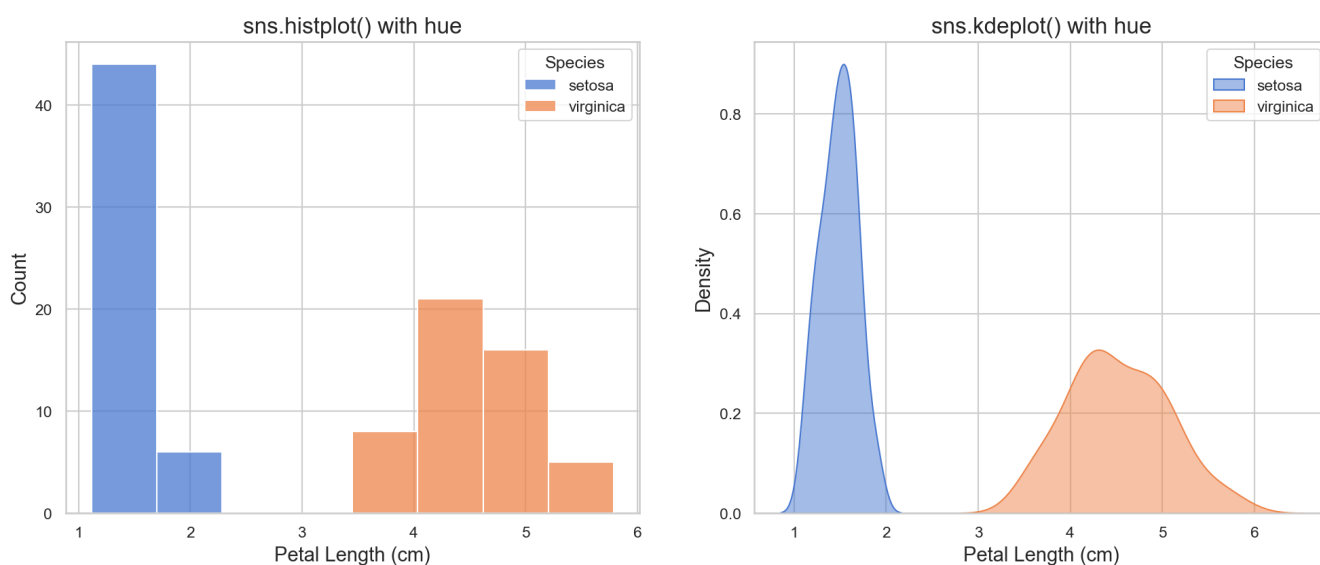


Рис. 5: Сравнение одномерных распределений классов через гистограмму и KDE

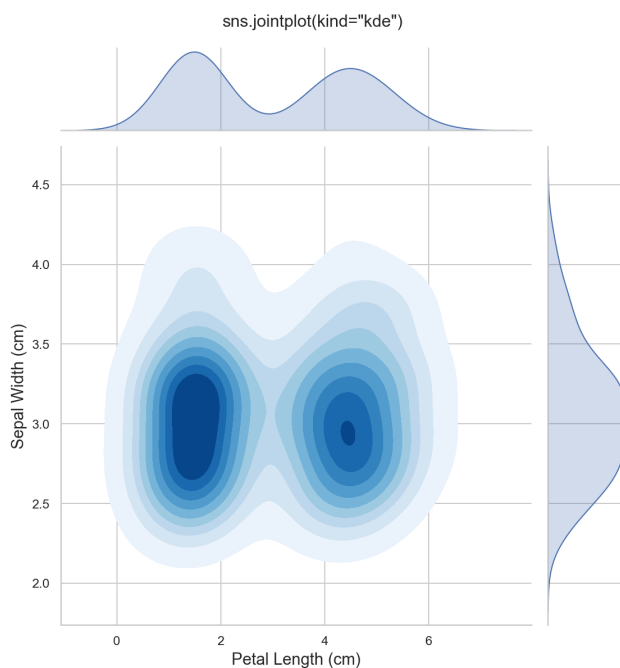


Рис. 6: Двумерное распределение плотности признаков через jointplot

1.3 Справочник параметров основных функций Seaborn

Ниже приведены ключевые аргументы, которые можно передавать в функции визуализации для кастомизации графиков.

1.3.1 Общие параметры для большинства графиков (`scatterplot`, `histplot`, `lineplot`, `barplot`)

- **data** — `DataFrame`, из которого берутся данные.
- **x, y** — Имена столбцов для осей абсцисс и ординат (или массивы/списки напрямую).
- **hue** — Столбец для цветовой группировки (разбивает данные по категориям или непрерывным значениям, окрашивая их в разные цвета).
- **palette** — Название цветовой палитры (например, `'viridis'`, `'coolwarm'`, `'Set1'`, `'magma'`, `'Blues_d'`).
- **ax** — Объект осей `Matplotlib` (`axs[0]`, `ax`), на котором нужно отрисовать график (важно при работе с `plt.subplots`).
- **alpha** — Прозрачность элементов (от 0 — полностью прозрачно, до 1 — глухой цвет). Помогает при наложении точек/слоев.

1.3.2 Специфичные параметры для конкретных графиков

1. Диаграммы рассеяния (`sns.scatterplot`)

- **style** — Имя столбца для группировки по форме маркеров (например, один класс будет круглыми точками, другой — крестиками).
- **size** — Изменение размера точек в зависимости от значений выбранного числового столбца.
- **sizes** — Диапазон размеров точек, передается кортежем, например: `sizes=(20, 200)`.
- **marker** — Принудительное задание формы для всех точек (например, `'x'`, `'o'`, `'s'` для квадратов).

2. Линейные графики и Тренды (`sns.lineplot` / `sns.regplot`)

- **style** (для `lineplot`) — Меняет тип линии (сплошная, пунктир, штрих-пунктир) для разных категорий.
- **markers** (для `lineplot`) — Если `True`, добавляет точки-маркеры на изгибы линий.
- **ci** или **errorbar** (для обоих) — Настройка отображения доверительного интервала (например, `errorbar='ci'` или `None`, чтобы убрать полупрозрачную тень ошибки).
- **scatter_kws** (для `regplot`) — Словарь параметров для настройки точек под капотом (например, `scatter_kws={'alpha': 0.3}`).
- **line_kws** (для `regplot`) — Словарь параметров для настройки самой линии тренда (например, `line_kws={'color': 'red', 'linewidth': 2}`).

3. Столбчатые диаграммы (`sns.barplot`)

- **orient** — Ориентация графика: `'v'` (вертикальный) или `'h'` (горизонтальный). Автоматически определяется по типам данных в `x` и `y`.
- **estimator** — Функция агрегации данных внутри баров (по умолчанию `np.mean`, но можно передать `np.median`, `np.sum` или `len`).
- **order** — Список, задающий строгий порядок отображения категорий на графике по именам.

4. Матрицы корреляций (`sns.heatmap`)

- **annot** — Если `True`, подписывает значения (числа) внутри каждой ячейки матрицы.
- **fmt** — Формат вывода чисел при `annot=True` (например, `".2f"` для двух знаков после запятой или `".0f"` для целых чисел).
- **linewidths** — Толщина разделительных линий между ячейками (например, `linewidths=.5`).
- **cbar** — Булево значение (`True/False`), показывать ли цветовую шкалу (`colorbar`) справа от матрицы.
- **vmin, vmax** — Минимальное и максимальное значения, к которым привязываются крайние цвета палитры (для корреляции ставят `vmin=-1`, `vmax=1`).

5. Распределения (`sns.histplot` / `sns.kdeplot`)

- **bins** (для `histplot`) — Количество столбцов (бакенов) на графике или их точные границы (например, `bins=20`).
- **kde** (для `histplot`) — Если `True`, автоматически накладывает плавную линию плотности распределения поверх столбцов гистограммы.
- **multiple** (для `histplot`) — Логика отображения нескольких классов при наличии `hue`. Варианты: `"layer"` (наложение), `"stack"` (столбцы растут друг на друге), `"dodge"` (столбцы стоят рядом).
- **fill** (для `kdeplot`) — Если `True`, закрашивает область под плавной кривой распределения.
- **bw_adjust** (для `kdeplot`) — Параметр сглаживания кривой. Менее 1 — график будет детальным и дерганым, больше 1 — максимально гладким.

6. Комплексные графики (sns.jointplot)

- **kind** — Тип отображения совместной зависимости по центру полотна. Варианты: "scatter" (точки), "kde" (топографические изолинии плотности), "hist" (двумерная гистограмма из квадратов), "reg" (точки + регрессия), "hex" (шестиугольные бакеры).
- **height** — Размер квадратного полотна в дюймах (задает общую пропорцию графика).
- **marginal_kws** — Словарь параметров для кастомизации дополнительных графиков распределения по краям осей (например, `marginal_kws={'fill': False, 'bins': 10}`).

```
1 # Scatterplot: со всеми параметрами кастомизации точек и отображения
2 sns.scatterplot(data=df, x='bmi', y='charges', hue='smoker', style='smoker', size='age',
3                 sizes=(20, 200), palette='viridis', alpha=0.7, marker='o', ax=ax[0])
4
5 # Lineplot: со стилизацией линий, маркерами на изгибах и отключенной тенью ошибки
6 sns.lineplot(data=df, x='time', y='signal', hue='sensor_type', style='sensor_type', markers=
7               True, errorbar=None, palette='Set1', alpha=0.9, ax=ax)
8
9 # Barplot: горизонтальный, с медианой вместо среднего и строгим порядком категорий
10 sns.barplot(data=df, x='score', y='genre', hue='platform', orient='h', estimator=np.median,
11              order=['Strategy', 'RPG', 'Racing'], palette='Blues_d', ax=ax)
12
13 # Heatmap: с числами, фиксированным диапазоном палитры и тонкими границами ячеек
14 sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', vmin=-1, vmax=1, linewidths
15              =0.5, cbar=True, ax=ax)
16
17 # Histplot: со столбцами друг на друге, встроенной линией KDE и фиксированным шагом
18 sns.histplot(data=df, x='petal_length', hue='species', multiple='stack', bins=30, kde=True,
19               palette='magma', alpha=0.6, ax=ax)
20
21 # Kdeplot: с гладкой заливкой под кривой и ручной регулировкой детализации графика
22 sns.kdeplot(data=df, x='petal_length', hue='species', fill=True, bw_adjust=0.7, palette='
23              crest', alpha=0.5, ax=ax)
24
25 # Jointplot: двумерная гексагональная сетка с кастомизацией боковых гистограмм
26 sns.jointplot(data=df, x='petal_length', y='sepal_width', kind='hex', height=8, palette='
27               Blues', marginal_kws={'bins': 15, 'fill': False})
```

2 Matplotlib & Seaborn: Глобальные настройки и Трюки

2.1 Оптимизация качества и параметров по умолчанию

Чтобы не прописывать размеры, шрифты и форматы для каждого отдельного полотна, в самом конце работы или сразу после импортов рекомендуется изменить глобальные значения по умолчанию с помощью словаря `plt.rcParams`.

```
1 # 1. Включение Retina-формата для идеальной четкости в Jupyter Notebook
2 # (В латехе пишем через знак вопроса вместо процента, чтобы не вешать компилятор)
3 ?config InlineBackend.figure_format = 'retina'
4
5 # 2. Быстрое переключение красивых тем и палитр в Seaborn
6 import seaborn as sns
7 import matplotlib.pyplot as plt
8
9 sns.set(style='darkgrid', palette='deep') # Стандартная аккуратная тема
10 # sns.set(style='whitegrid', palette='deep') # Белая тема (удобно для печати)
11 # sns.set(style='darkgrid', palette='rocket') # Темная контрастная тема
12
13 # 3. Установка единых параметров отображения через rcParams
14 plt.rcParams['figure.figsize'] = 8, 5 # Дефолтный размер полотна (ширина, высота)
15 plt.rcParams['font.size'] = 12 # Базовый размер шрифта для всех подписей
16 plt.rcParams['savefig.format'] = 'pdf' # Экспорт графиков в векторный PDF по умолчанию
```

Листинг 1: Глобальная настройка графиков и стилей

2.2 Разбор базовых настроек полотна

```
1 # Быстрое изменение размера конкретного графика перед отрисовкой
2 plt.figure(figsize=(9, 6))
3
4 plt.plot(x, y)
5
6 # Обязательные элементы хорошего графика: подписи и заголовки
7 plt.title("Название графика")
8 plt.xlabel("Подпись горизонтальной оси X")
9 plt.ylabel("Подпись вертикальной оси Y")
10
11 plt.show()
```

Листинг 2: Базовый шаблон оформления графика

3 Библиотека Pandas: Ввод-Вывод данных (I/O)

3.1 Полное семейство функций чтения (read_*)

```
1 import pandas as pd
2 import numpy as np
3
4 # 1. Текстовые форматы с разделителями
5 df_csv = pd.read_csv('data.csv', sep=',', encoding='utf-8', index_col=0)
6 df_txt = pd.read_table('data.txt', sep='\t')
7
8 # 2. Табличные процессоры и документы
9 df_excel = pd.read_excel('data.xlsx', sheet_name='Sheet1')
10
11 # 3. Веб-страницы и структурированный обмен (ищет теги <table>)
12 df_html_list = pd.read_html('https://example.com/table_page.html')
13 df_json = pd.read_json('data.json')
14
15 # 4. Реляционные базы данных (SQL Queries)
16 # con - активное соединение, созданное через sqlalchemy
17 df_sql = pd.read_sql("SELECT * FROM users_table", con=engine)
18
19 # 5. Буфер обмена (быстрый импорт скопированных руками ячеек)
20 df_clipboard = pd.read_clipboard()
21
22 # 6. Высокопроизводительные бинарные форматы (Big Data)
23 df_parquet = pd.read_parquet('data.parquet')
24 df_feather = pd.read_feather('data.feather')
25 df_hdf = pd.read_hdf('data.h5', key='df')
```

Листинг 3: Загрузка внешних табличных данных в DataFrame

3.2 Полное семейство функций экспорта (to_*)

```
1 # Экспорт без сохранения технического порядкового индекса
2 df.to_csv('output.csv', index=False, sep=';')
3 df.to_excel('output.xlsx', sheet_name='Summary', index=False)
4 df.to_json('output.json', orient='records')
5
6 # Запись в реляционную таблицу (if_exists: 'fail', 'replace', 'append')
7 df.to_sql('target_table', con=engine, if_exists='replace', index=False)
8
9 # Бинарный быстрый экспорт
10 df.to_parquet('output.parquet')
11 df.to_feather('output.feather')
```

Листинг 4: Сохранение DataFrame на диск

4 Pandas: Серии, Структура и Продвинутая Индексация

4.1 Работа с одномерными массивами (Series)

```
1 # Series - одномерный массив с явными метками (индексами)
2 s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'], name='Age')
3
4 print(s.index) # Доступ к меткам (Index(['a', 'b', 'c', 'd']))
5 print(s.values) # Доступ к чистому numpy-массиву array([10, 20, 30, 40])
6
7 # Доступ к элементам
8 print(s['b']) # По метке -> 20
9 print(s[1]) # По позиции -> 20
```

Листинг 5: Создание и свойства Series

4.2 Создание и базовое исследование DataFrame

```
1 df = pd.DataFrame({
2     'A': [1, 2, 3, 4],
3     'B': [10.5, np.nan, 30.2, 40.0],
4     'C': ['foo', 'bar', 'foo', 'baz']
5 }, index=['row1', 'row2', 'row3', 'row4'])
6
7 df.shape # Размерность: (строки, колонки)
8 df.columns # Список/индекс названий столбцов
9 df.dtypes # Сводка типов данных по столбцам
10 df.info() # Подробная техническая информация и память
11 df.describe() # Статистика: mean, std, min, max, квантили
```

Листинг 6: Инициализация и атрибуты

4.3 Различия loc и iloc при работе со срезами [:]

```
1 # 1. .loc - выборка по ЯВНЫМ текстовым/числовым меткам
2 # ВНИМАНИЕ: Правая граница ВКЛЮЧАЕТСЯ! ('row3' попадет в выборку)
3 df_loc = df.loc['row1':'row3', ['A', 'C']]
4
5 # 2. .iloc - выборка по ПОРЯДКОВЫМ номерам позиций (от 0 до N-1)
6 # Правая граница НЕ ВКЛЮЧАЕТСЯ (классический питоновский срез)
7 df_iloc = df.iloc[0:2, 0:2] # Строки 0,1; колонки 0,1
8
9 # 3. Быстрый выбор через квадратные скобки без методов:
10 df[:2] # Порядковый срез строк (эквивалентно iloc[0:2])
11 df['A'] # Выбор одного столбца (возвращает Series)
12 df[['A', 'C']] # Выбор подмножества столбцов (возвращает DataFrame)
```

Листинг 7: Диапазоны и срезы строк

4.4 Продвинутая фильтрация по условиям (Булевы маски)

```
1 # Операторы сравнения: ==, !=, >, <, >=, <=
2 mask = df['A'] >= 2
3
4 # Логические побитовые операторы: & (И), | (ИЛИ), ~ (НЕ)
5 # Обязательно оборачивать каждое условие в круглые скобки (!)
6 filtered = df[(df['A'] > 1) & (df['C'] == 'foo')]
7 not_foo = df[~(df['C'] == 'foo')] # Отрицание (все, кроме foo)
8
9 # Встроенные методы фильтрации
10 df[df['C'].isin(['foo', 'baz'])] # Значение входит в список
11 df[df['B'].isna()] # Нахождение пропусков (NaN)
12 df[df['B'].notna()] # Нахождение заполненных ячеек
```

Листинг 8: Логические сравнения и операторы

5 Pandas: Трансформация, Группировки и Объединения

5.1 Применение функций: map, apply и applymap

```
1 # 1. map() - применяется ТОЛЬКО к Series (столбцу) для замены значений по словарю или функции
2 gender_dict = {'foo': 'X', 'bar': 'Y', 'baz': 'Z'}
3 df['C_mapped'] = df['C'].map(gender_dict)
4 df['A_squared'] = df['A'].map(lambda x: x**2)
5
6 # 2. apply() - применяется к Series или к DataFrame вдоль указанной оси (axis)
7 # На Series: трансформация каждого элемента функции
8 df['B_logged'] = df['B'].apply(np.log)
9
10 # На DataFrame: axis=0 (по колонкам, дефолт), axis=1 (по строкам)
11 # Пример: считаем размах (max - min) для числовых колонок
12 numeric_df = df[['A', 'B']]
13 col_ranges = numeric_df.apply(lambda col: col.max() - col.min(), axis=0)
14 row_sums = numeric_df.apply(lambda row: row['A'] + row['B'], axis=1)
15
16 # 3. applymap() / map() для всего DF - поэлементное применение ко всей таблице
17 string_df = df[['C']].astype(str).map(lambda val: val.upper())
```

Листинг 9: Преобразование данных в столбцах и таблицах

5.2 Агрегация и Группировка: groupby и agg

```
1 # Данные для примера
2 sales = pd.DataFrame({
3     'Store': ['S1', 'S1', 'S2', 'S2', 'S1'],
4     'Item': ['Apple', 'Orange', 'Apple', 'Apple', 'Orange'],
5     'Revenue': [100, 150, 200, 50, 120],
6     'Quantity': [10, 15, 8, 2, 11]
7 })
8
9 # Простая группировка с одной встроенной функцией (сумма, среднее и т.д.)
10 grouped_sum = sales.groupby('Store')['Revenue'].sum()
11
12 # Сложная агрегация через метод .agg()
13 # Позволяет вешать разные функции на разные колонки одновременно
14 agg_res = sales.groupby('Store').agg({
15     'Revenue': ['sum', 'mean'], # Для Revenue считаем сумму и среднее
16     'Quantity': lambda x: x.max() - x.min() # Кастомная лямбда (размах)
17 })
18
19 # Группировка по нескольким ключам (Мультииндекс)
20 multi_group = sales.groupby(['Store', 'Item']).mean()
21 # Сброс мультииндекса обратно в плоскую таблицу
22 flat_df = multi_group.reset_index()
```

Листинг 10: Разделение, расчет метрик и сборка (Split-Apply-Combine)

5.3 Работа с пропусками: fillna и dropna

```
1 # 1. fillna() - заполнение пропусков константой или метрикой
2 df['B_filled'] = df['B'].fillna(0) # Замена всех NaN на ноль
3 df['B_median'] = df['B'].fillna(df['B'].median()) # Замена на медиану столбца
4
5 # Использование inplace=True изменяет исходный объект, а не возвращает копию
6 df['B'].fillna(method='ffill', inplace=True) # Заполнение предыдущим значением (forward fill)
7
8 # 2. dropna() - удаление строк или столбцов с пропусками
9 df_clean_rows = df.dropna(axis=0) # Удалить строки, если в них есть хоть один NaN
10 df_clean_cols = df.dropna(axis=1) # Удалить столбцы, если в них есть NaN
```

Листинг 11: Обработка неопределенных значений (NaN)

5.4 Слияние и Склейка таблиц: concat, merge, join

```
1 df1 = pd.DataFrame({'K': ['b', 'd'], 'X': [1, 2]})
2 df2 = pd.DataFrame({'K': ['b', 'c'], 'Y': [3, 4]})
3
4 # 1. pd.concat() - жесткая склейка вдоль оси «стык в стык»
5 # axis=0: пристыковывает строки снизу увеличивает( длину)
6 concat_v = pd.concat([df1, df2], axis=0, ignore_index=True)
7 # axis=1: пристыковывает колонки справа по индексам строк увеличивает( ширину)
8 concat_h = pd.concat([df1, df2], axis=1)
9
10 # 2. pd.merge() - полноценный аналог SQL JOIN по ключевым столбцам
11 # how может быть: 'inner' дефолт(), 'outer' полный(), 'left', 'right'
12 merged_inner = pd.merge(df1, df2, on='K', how='inner') # Совпадения по ключу 'K'
13 merged_left = pd.merge(df1, df2, on='K', how='left') # Все из левой + хвост справа
14
15 # 3. df.join() - объединение по ИНДЕКСАМ строк а( не по колонкам)
16 # Удобно, когда ключи вынесены в index таблиц
17 df1.set_index('K', inplace=True)
18 df2.set_index('K', inplace=True)
19 joined_df = df1.join(df2, how='outer')
```

Листинг 12: Комбинирование нескольких DataFrame

5.5 Применение функций: map, apply и applymap

```
1 # 1. map() - применяется ТОЛЬКО к Series (столбцу) для замены значений по словарю или функции
2 gender_dict = {'foo': 'X', 'bar': 'Y', 'baz': 'Z'}
3 df['C_mapped'] = df['C'].map(gender_dict)
4 df['A_squared'] = df['A'].map(lambda x: x**2)
5
6 # 2. apply() - применяется к Series или к DataFrame вдоль указанной оси (axis)
7 # На Series: трансформация каждого элемента функции
8 df['B_logged'] = df['B'].apply(np.log)
9
10 # На DataFrame: axis=0 (по колонкам, дефолт), axis=1 (по строкам)
11 # Пример: считаем размах (max - min) для числовых колонок
12 numeric_df = df[['A', 'B']]
13 col_ranges = numeric_df.apply(lambda col: col.max() - col.min(), axis=0)
14 row_sums = numeric_df.apply(lambda row: row['A'] + row['B'], axis=1)
15
16 # 3. applymap() / map() для всего DF - поэлементное применение ко всей таблице
17 string_df = df[['C']].astype(str).map(lambda val: val.upper())
```

Листинг 13: Преобразование данных в столбцах и таблицах

5.6 Агрегация, Математика и Группировка: groupby и agg

```
1 # Данные для примера
2 sales = pd.DataFrame({
3     'Store': ['S1', 'S1', 'S2', 'S2', 'S1'],
4     'Item': ['Apple', 'Orange', 'Apple', 'Apple', 'Orange'],
5     'Revenue': [100, 150, 200, 50, 120],
6     'Quantity': [10, 15, 8, 2, 11]
7 })
8
9 # Базовые встроенные методы агрегации работают( на Series и DataFrame)
10 print(sales['Revenue'].sum()) # Сумма элементов столбца
11 print(sales['Revenue'].mean()) # Среднее арифметическое значение
12 print(sales['Revenue'].min()) # Минимальное значение
13 print(sales['Revenue'].max()) # Максимальное значение
14 print(sales['Revenue'].count()) # Количество НЕ пустых (non-NaN) значений
15 print(sales['Revenue'].std()) # Стандартное отклонение (Standard Deviation)
16 print(sales['Revenue'].var()) # Дисперсия (Variance)
17 print(sales['Revenue'].median()) # Медиана
18
19 # Простая группировка с одной встроенной функцией сумма(, среднее и тд..)
20 grouped_sum = sales.groupby('Store')['Revenue'].sum()
21
```

```

22 # Сложная агрегация через метод .agg()
23 # Позволяет вешать разные функции на разные колонки одновременно
24 agg_res = sales.groupby('Store').agg({
25     'Revenue': ['sum', 'mean'],          # Для Revenue считаем сумму и среднее
26     'Quantity': lambda x: x.max() - x.min() # Кастомная лямбда размах()
27 })
28
29 # Группировка по нескольким ключам Мультииндекс()
30 multi_group = sales.groupby(['Store', 'Item']).mean()
31 # Сброс мультииндекса обратно в плоскую таблицу
32 flat_df = multi_group.reset_index()

```

Листинг 14: Разделение, расчет метрик и сборка данных

5.7 Исследование матриц, статистики и удаление: drop и describe

```

1 # 1. Сводная статистика признаков через .describe()
2 # Возвращает count, mean, std, min, max и квантили (25%, 50%, 75%)
3 stats = sales.describe()
4 stats_all = sales.describe(include='all') # Включает также текстовые колонки (unique, top,
      freq)
5
6 # 2. Удаление строк и столбцов через .drop()
7 # axis=1 (или columns=['...']) - удаляет указанные столбцы
8 df_dropped_cols = sales.drop(['Quantity', 'Item'], axis=1)
9 # axis=0 (или index=[]) - удаляет строки по их индексам/меткам
10 df_dropped_rows = sales.drop([1, 3], axis=0)

```

Листинг 15: Методы анализа структуры и очистки

5.8 Корреляции и матрицы сопряженности: corr, cov

```

1 # 1. Матрица корреляции .corr() (по умолчанию метод Пирсона)
2 # Показывает силу линейной зависимости между парами колонок от -1 до 1
3 matrix_corr = sales[['Revenue', 'Quantity']].corr()
4
5 # Альтернативные методы корреляции для нелинейных или ранговых связей
6 matrix_spearman = sales[['Revenue', 'Quantity']].corr(method='spearman') # Спирмен
7 matrix_kendall = sales[['Revenue', 'Quantity']].corr(method='kendall') # Кендалл
8
9 # 2. Матрица ковариации .cov()
10 # Показывает направление связи (масштабируется в зависимости от дисперсии признаков)
11 matrix_cov = sales[['Revenue', 'Quantity']].cov()
12
13 # 3. Как на это смотреть: обычно выводят тепловую карту через seaborn
14 # import seaborn as sns
15 # sns.heatmap(matrix_corr, annot=True, cmap='coolwarm')

```

Листинг 16: Расчет взаимосвязей числовых признаков

6 Сравнение библиотек: Pandas vs Polars

Перед началом работы важно понимать ключевые архитектурные различия между классическим инструментом Pandas и современным движком Polars.

6.1 В чем Polars лучше Pandas (Плюсы Polars)

- **Скорость и многопоточность:** Polars написан на языке Rust и по умолчанию параллелизирует вычисления, используя все доступные ядра процессора. Pandas по своей природе работает строго в один поток.
- **Ленивые вычисления (Lazy Evaluation):** Polars умеет строить граф запроса (`LazyFrame`) и оптимизировать его перед реальным выполнением: он автоматически выкидывает ненужные колонки и объединяет фильтры, не загружая в оперативную память лишний хлам.
- **Эффективное управление памятью:** Благодаря использованию стрелочной памяти (Apache Arrow), Polars расходует в разы меньше RAM и избегает лишнего дублирования данных при мелких операциях.
- **Предсказуемый синтаксис:** В Polars полностью отсутствует путаница с индексами и скрытыми копиями таблиц, а значит, вы никогда не столкнетесь с раздражающим предупреждением `SettingWithCopyWarning`.

6.2 В чем Polars хуже Pandas (Минусы Polars)

- **Полное отсутствие индексов:** В Polars нет концепции `.index`. Строки всегда имеют только базовую порядковую нумерацию от 0. Если в задаче критически необходим старый индекс, его приходится принудительно сохранять как отдельную колонку.
- **Меньше готовых интеграций:** Огромное количество старых библиотек для машинного обучения (scikit-learn) и визуализации (seaborn, matplotlib) исторически требуют на вход строго `pandas.DataFrame`. Из-за этого в коде приходится дописывать костыльный метод `.to_pandas()`.
- **Максимально строгая типизация:** Polars не разрешит вам смешивать разные типы данных внутри одной колонки или проводить сравнения типа `int == float` без явного ручного приведения типов через метод `.cast()`.

```
1 import pandas as pd
2 import polars as pl
3
4 # 1. Из Polars в Pandas
5 df_pd = df_pl.to_pandas()
6
7 # 2. Из Pandas в Polars
8 df_pl = pl.from_pandas(df_pd)
```

Листинг 17: Конвертация между Pandas и Polars

6.3 Ключевые фишки и синтаксические различия

В отличие от Pandas, где выборка и фильтрация часто смешиваются в перегруженных квадратных скобках `[]`, Polars разделяет эти операции на строгие оптимизированные методы.

```
1 # 1. Выбор колонок через .select() вместо df[['col1', 'col2']]
2 # Работает быстрее пандаса, так как не копирует данные в памяти
3 df_pl.select(['apple_id', 'price'])
4
5 # 2. Фильтрация строк через .filter() вместо df[df['price'] > 100]
6 df_pl.filter(pl.col('price') > 100)
7
8 # 3. Добавление индекса строк через .with_row_index()
9 # Так как в Polars нет .index, этот метод создает честный столбец с ID
10 df_indexed = df_pl.with_row_index(name='id')
11
12 # 4. Моментальный подсчет пропусков через .null_count()
13 # Возвращает количество Null-значений для каждого столбца без тяжелых циклов
14 null_summary = df_pl.null_count()
```

Листинг 18: Специфичные фишки и методы Polars

7 Библиотека Polars: Ввод-Вывод данных (I/O)

7.1 Полное семейство функций чтения (read_* и scan_*)

Polars разделяет чтение на Eager (мгновенная загрузка в память через read) и Lazy (сканирование файла и построение графа вычислений через scan). На больших данных scan гарантирует, что программа не вылетит по памяти (Out-Of-Memory).

```
1 import polars as pl
2 import numpy as np
3
4 # 1. Текстовые форматы (Eager и Lazy)
5 df_csv = pl.read_csv('data.csv', separator=',', encoding='utf-8')
6 lazy_csv = pl.scan_csv('data.csv', separator=',')
7
8 # 2. Табличные процессоры Excel (требуется xlsx2csv или fastexcel)
9 df_excel = pl.read_excel('data.xlsx', sheet_name='Sheet1')
10
11 # 3. Высокопроизводительные бинарные форматы (Big Data)
12 df_parquet = pl.read_parquet('data.parquet')
13 lazy_parquet = pl.scan_parquet('data.parquet')
14
15 df_ipc = pl.read_ipc('data.arrow') # Формат Arrow/Feather
16 lazy_ipc = pl.scan_ipc('data.arrow')
17
18 # 4. Базы данных и JSON формат
19 df_json = pl.read_json('data.json')
20 # Считывание из SQL требует активного URI подключения к БД
21 df_sql = pl.read_database("SELECT * FROM table", connection='postgres://...')
```

Листинг 19: Загрузка внешних табличных данных в Polars

7.2 Полное семейство функций экспорта (write_*)

```
1 # Экспорт объектов DataFrame в файлы (в Polars нет индексов, сохранять их не нужно)
2 df.write_csv('output.csv', separator=';')
3 df.write_excel('output.xlsx', sheet_name='Summary')
4 df.write_json('output.json')
5
6 # Высокопроизводительный бинарный экспорт
7 df.write_parquet('output.parquet')
8 df.write_ipc('output.arrow')
```

Листинг 20: Сохранение таблиц Polars на диск

8 Polars: Структура данных и Способы Индексации

8.1 Работа с одномерными массивами (Series)

Объект Series представляет собой именованный одномерный массив фиксированного типа. Индексов по меткам (как в Pandas) здесь нет. Доступ к элементам осуществляется строго по их порядковому номеру.

```
1 s = pl.Series('Age', [10, 20, 30, 40], dtype=pl.Int64)
2
3 print(s.name)      # Получить имя серии
4 print(s.dtype)     # Получить тип данных
5 print(s[1])        # Доступ по порядковой позиции -> 20
```

Листинг 21: Создание и свойства Series в Polars

8.2 Создание и базовое исследование DataFrame

DataFrame инициализируется аналогично словарю. Атрибуты .shape, .columns, .dtypes возвращают размеры и структуру. Просмотр срезов строк осуществляется через методы .head(), .tail() и .sample().

```
1 df = pl.DataFrame({
2     'A': [1, 2, 3, 4],
3     'B': [10.5, None, 30.2, 40.0], # Вместо np.nan используется None
4     'C': ['foo', 'bar', 'foo', 'baz']
5 })
6
7 print(df.shape)
8 print(df.columns)
9 print(df.dtypes)
10 print(df.head(2))
11 print(df.sample(2))
```

Листинг 22: Инициализация и атрибуты DataFrame

8.3 Выборка данных и срезы (Вместо loc и iloc)

В Polars нет методов .loc и .iloc. Доступ к строкам и колонкам реализуется через метод .select() (для столбцов), метод .filter() (для условий строк) или через стандартные срезы матриц [строки, колонки].

```
1 # Выборка колонок через контекст выражений (самый быстрый и правильный путь)
2 df_cols = df.select(['A', 'C'])
3 df_cols_expr = df.select(pl.col('A'), pl.col('C'))
4
5 # Прямые срезы по позициям (как в numpy / iloc)
6 df_slice = df[0:2, :]          # Первые две строки, все колонки
7 df_cell = df[2, 'B']           # Элемент на пересечении 2-й строки и колонки 'B'
```

Листинг 23: Диапазоны строк и извлечение признаков

8.4 Продвинутая фильтрация по условиям

Фильтрация строк выполняется через метод .filter() с использованием выражений pl.col(). Побитовые операторы связывают условия: & (И), | (ИЛИ), ~ (НЕ). Каждое условие обязано быть внутри круглых скобок. Нахождение пустых ячеек выполняется методами .is_null() и .is_not_null().

```
1 # Фильтрация по одному условию
2 high_values = df.filter(pl.col('A') >= 2)
3
4 # Сложные составные условия в круглых скобках
5 filtered = df.filter((pl.col('A') > 1) & (pl.col('C') == 'foo'))
6 not_foo = df.filter(~(pl.col('C') == 'foo'))
7
8 # Проверка на пропуски и вхождение в список элементов
9 isin_mask = df.filter(pl.col('C').is_in(['foo', 'baz']))
10 null_mask = df.filter(pl.col('B').is_null())
11 not_null_mask = df.filter(pl.col('B').is_not_null())
```

Листинг 24: Логический выбор строк в Polars

9 Polars: Трансформация, Группировки, Математика и Объединения

9.1 Применение выражений и кастомных функций: `map_elements`

Вместо `map/apply` в Polars используется контекст выражений с методом `.map_elements()`. Для простых замен по словарю служит встроенный эффективный метод `.replace_strict()` (или `.replace`).

```
1 # 1. Замена значений по словарю (вместо .map())
2 gender_dict = {'foo': 'X', 'bar': 'Y', 'baz': 'Z'}
3 df = df.with_columns(pl.col('C').replace_strict(gender_dict, default=None).alias('C_replaced'
4 ))
5
6 # 2. Применение лямбда-функций к ячейкам (вместо .apply())
7 df = df.with_columns(pl.col('A').map_elements(lambda x: x**2, return_dtype=pl.Int64).alias('
8 A_squared'))
9
10 # 3. Математические трансформации строк "на лету" без лямбд
11 df = df.with_columns(pl.col('B').log().alias('B_logged'))
```

Листинг 25: Преобразование данных и выражений

9.2 Агрегация, Математика и Группировка: `group_by` и `agg`

Группировка в Polars пишется через метод `.group_by()` (с нижним подчеркиванием). Внутри метода `.agg()` передается список агрегирующих выражений. Названия встроенных математических функций совпадают с pandas.

```
1 sales = pl.DataFrame({
2     'Store': ['S1', 'S1', 'S2', 'S2', 'S1'],
3     'Item': ['Apple', 'Orange', 'Apple', 'Apple', 'Orange'],
4     'Revenue': [100, 150, 200, 50, 120],
5     'Quantity': [10, 15, 8, 2, 11]
6 })
7
8 # Базовые агрегации для всей таблицы
9 print(sales.select(pl.col('Revenue').sum()))
10 print(sales.select(pl.col('Revenue').mean()))
11 print(sales.select(pl.col('Revenue').min()))
12 print(sales.select(pl.col('Revenue').max()))
13 print(sales.select(pl.col('Revenue').count()))
14 print(sales.select(pl.col('Revenue').std()))
15 print(sales.select(pl.col('Revenue').var()))
16
17 # Группировка по ключу и расчет агрегаций
18 agg_res = sales.group_by('Store').agg(
19     pl.col('Revenue').sum().alias('Total_Revenue'),
20     pl.col('Revenue').mean().alias('Mean_Revenue'),
21     pl.col('Quantity').max().alias('Max_Qty')
22 )
```

Листинг 26: Расчет базовых метрик и группировка данных

9.3 Исследование матриц, статистики и удаление: `drop` и `describe`

```
1 # 1. Вывод общей описательной статистики числовых признаков
2 print(sales.describe())
3
4 # 2. Удаление колонок по названиям из DataFrame
5 df_dropped = sales.drop(['Quantity', 'Item'])
```

Листинг 27: Методы анализа структуры и очистки матриц

9.4 Расчет матриц корреляции и ковариации

В Polars нет метода `.corr()` у самого `DataFrame`. Корреляция Пирсона или Спирмена и ковариация считаются через функции `pl.corr()` и `pl.cov()` внутри метода `.select()`.

```
1 # Расчет корреляции Пирсона между парами признаков
2 corr_pearson = sales.select(pl.corr('Revenue', 'Quantity', method='pearson'))
3
4 # Расчет корреляции Спирмена для ранговых зависимостей
```

```

5 corr_spearman = sales.select(pl.corr('Revenue', 'Quantity', method='spearman'))
6
7 # Расчет матрицы ковариации признаков
8 covariance = sales.select(pl.cov('Revenue', 'Quantity'))

```

Листинг 28: Расчет взаимосвязей признаков

9.5 Работа с пропусками: fill_null и drop_null

Параметры и функции для работы с пропущенными значениями в Polars имеют окончание `_null` (а не `_na`).

```

1 # 1. Заполнение пропусков константой или стратегией (медианой, ffill)
2 df_filled = df.with_columns(pl.col('B').fill_null(0))
3 df_median = df.with_columns(pl.col('B').fill_null(pl.col('B').median()))
4 df_ffill = df.with_columns(pl.col('B').fill_null(strategy='forward'))
5
6 # 2. Удаление строк, если в них содержится Null
7 df_clean = df.drop_nulls() # Удаляет строки, где есть хоть один null

```

Листинг 29: Обработка неопределенных ячеек Null

9.6 Слияние и Склейка таблиц: concat и join

Объединение таблиц в Polars реализуется через глобальную функцию `pl.concat()` и метод `.join()`.

```

1 df1 = pl.DataFrame({'K': ['b', 'd'], 'X': [1, 2]})
2 df2 = pl.DataFrame({'K': ['b', 'c'], 'Y': [3, 4]})
3
4 # 1. Склейка таблиц стык в стык через pl.concat
5 # how может быть 'vertical' (наращивание строк) или 'horizontal' (наращивание колонок)
6 concat_v = pl.concat([df1, df2], how='vertical')
7
8 # 2. Объединение таблиц по ключевой колонке (JOIN)
9 # how может быть: 'inner', 'left', 'right', 'outer', 'semi', 'anti'
10 merged_inner = df1.join(df2, on='K', how='inner')
11 merged_left = df1.join(df2, on='K', how='left')

```

Листинг 30: Комбинирование нескольких DataFrame

10 NumPy для Machine Learning: Векторизация и Матрицы

10.1 Создание массивов и генерация данных

В машинном обучении инициализация весов, создание признаков и генерация случайных распределений делаются через базовые конструкторы матриц.

```
1 import numpy as np
2
3 # Создание векторов и матриц вручную
4 weight_vector = np.array([0.5, -1.2, 3.4])
5 X_toy = np.array([[1, 2], [3, 4], [5, 6]])
6
7 # Инициализация нулями (начальный вектор весов модели)
8 w_zeros = np.zeros(shape=(10, 1))
9
10 # Инициализация единицами (например для добавления bias-колонки)
11 bias_column = np.ones(shape=(X_toy.shape[0], 1))
12
13 # Создание последовательностей точек и шагов
14 grid_linear = np.linspace(0, 1, 100) # 100 точек от 0 до 1 включительно
15 range_steps = np.arange(0, 10, 0.2) # от 0 до 10 с шагом 0.2
```

Листинг 31: Способы создания и инициализации массивов

10.2 Генерация случайных величин и воспроизводимость

Для разбиения выборки, инициализации весов нейросетей или стохастического градиентного спуска критически важна генерация случайных чисел с фиксацией seed.

```
1 # Фиксация случайности для воспроизводимости ML-экспериментов
2 np.random.seed(42)
3
4 # Выборка случайных индексов объектов (например для батчей в SGD)
5 batch_indices = np.random.choice(100, size=16, replace=False)
6
7 # Инициализация весов из стандартного нормального распределения
8 W_normal = np.random.randn(5, 3)
9
10 # Генерация шума из равномерного распределения на отрезке от 0 до 1
11 noise = np.random.rand(10)
```

Листинг 32: Случайные распределения

10.3 Манипуляции с формами матриц (Reshaping и Склеивание)

Частая задача в ML — изменить размерность тензора (например вытянуть картинку в вектор) или склеить признаки объектов с вектором единиц.

```
1 # Изменение размерности без копирования данных в памяти
2 # Переход от трехмерного тензора к двумерной матрице признаков
3 X_flat = X_toy.reshape(3, -1) # Минус один вычисляет ось автоматически
4
5 # Добавление эффективной фиктивной оси (вектор в матрицу-столбец)
6 v = np.array([1, 2, 3])
7 v_col = v[:, np.newaxis] # Из формы (3,) получаем матрицу (3, 1)
8
9 # Объединение матриц (добавление новых признаков к объектам)
10 # hstack это склеивание по колонкам, vstack это склеивание по строкам
11 X_with_bias = np.hstack((bias_column, X_toy))
```

Листинг 33: Изменение формы и объединение матриц

10.4 Векторизация, Агрегаты и Матричные операции

Вместо циклов в Python для подсчета потерь (MSE, LogLoss) или предсказания линейной модели всегда используются быстрые векторизованные операции.

```
1 # 1. Скалярное матричное произведение через оператор собачки
2 # Вычисление предсказаний линейной регрессии для всей выборки разом
3 predictions = X_toy @ weight_vector[:2]
```

```

4
5 # 2. Поэлементные операции (Broadcasting)
6 # Применение сигмoиды ко всем выходам линейной модели разом
7 probabilities = 1 / (1 + np.exp(-predictions))
8
9 # 3. Агрегаты по осям ноль ( для колонок, единица для строк)
10 # Вычисление среднего значения каждого признака по всей выборке
11 feature_means = X_toy.mean(axis=0)
12
13 # 4. Быстрое векторизованное вычисление метрики MSE
14 y_true = np.array([1.0, 0.0, 1.0])
15 y_pred = np.array([0.9, 0.1, 0.8])
16 mse_loss = np.mean((y_true - y_pred) ** 2)
17
18 # 5. Фильтрация по условию замена( циклов и условий)
19 # Реализация функции активации ReLU замена( отрицательных значений на ноль)
20 relu_outputs = np.maximum(0, predictions)
21
22 # Получение индексов объектов где модель ошиблась сильнее порога
23 outliers = np.where(np.abs(y_true - y_pred) > 0.5)[0]

```

Листинг 34: Математические вычисления и метрики

11 Линейная регрессия и библиотека sklearn

11.1 1. Теоретическая база и Разведочный анализ (EDA)

- **Линейная регрессия:** Модель предсказания непрерывного таргета: $a(x) = \sum_{j=1}^d w_j x^j + w_0 = \langle w, x \rangle + w_0$. Обучение заключается в минимизации среднеквадратичной ошибки (MSE): $Q(w, X) = \frac{1}{\ell} \sum_{i=1}^{\ell} (\langle w, x_i \rangle - y_i)^2 \rightarrow \min_w$.
- **Мультиколлинеарность:** Наличие сильной линейной зависимости между признаками. Приводит к неустойчивости матрицы $(X^T X)^{-1}$, взрыву весов w по модулю и потере обобщающей способности (переобучению).
- **Борьба с коллинеарностью через корреляцию:** Перед обучением строят матрицу корреляций Пирсона и удаляют избыточные признаки.

```
1 import pandas as pd
2 import numpy as np
3
4 # Вычисление матрицы корреляций для числовых признаков
5 corr_matrix = df.corr(numeric_only=True)
6
7 # Выделение целевой переменной и оценка линейной связи с таргетом
8 target_corr = corr_matrix['price'].sort_values(ascending=False)
9
10 # Поиск пар признаков с избыточной корреляцией (более 0.85)
11 upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
12 to_drop = [column for column in upper_tri.columns if any(upper_tri[column].abs() > 0.85)]
13 df_filtered = df.drop(columns=to_drop)
```

Листинг 35: Анализ корреляций и фильтрация мультиколлинеарных признаков

11.2 2. Продвинутая обработка признаков (Feature Engineering)

Разделение данных на обучающую и тестовую выборки должно происходить строго до любых трансформаций признаков, чтобы избежать утечки данных (data leak).

```
1 from sklearn.model_split import train_test_split
2
3 X = df_filtered.drop(columns=['price'])
4 y = df_filtered['price']
5
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Листинг 36: Разделение данных и базовый синтаксис fit/transform

1. **Масштабирование числовых признаков (StandardScaler):** Сдвигает среднее признака к 0 и масштабирует дисперсию к 1: $x_{new} = \frac{x - \mu}{\sigma}$. Это критически важно для корректной работы регуляризации (иначе штраф будет зависеть от единиц измерения).
2. **Кодирование категориальных признаков (OneHotEncoder):** Преобразует категорию в набор бинарных векторов. Во избежание линейной зависимости (дамми-ловушки) в линейных моделях активируют параметр `drop='first'`.
3. **Дискретизация непрерывных величин (KBinsDiscretizer):** Разбивает непрерывный признак на интервалы (бины) и кодирует их. Помогает линейной модели улавливать нелинейные зависимости (например, нелинейное влияние года постройки здания).

```
1 from sklearn.preprocessing import StandardScaler, OneHotEncoder, KBinsDiscretizer
2
3 # Масштабирование: fit считается ТОЛЬКО на train, на test только transform!
4 scaler = StandardScaler()
5 X_train_scaled = scaler.fit_transform(X_train[['sqft']])
6 X_test_scaled = scaler.transform(X_test[['sqft']])
7
8 # Дискретизация на 5 интервалов с One-Hot кодированием исхода
9 discretizer = KBinsDiscretizer(n_bins=5, encode='onehot-dense', strategy='quantile')
10 X_train_bins = discretizer.fit_transform(X_train[['year_built']])
```

Листинг 37: Пример точечного использования трансформеров

11.3 3. Преобразование целевой переменной

Если распределение целевой переменной асимметрично (обладает «тяжелым хвостом»), линейная регрессия работает неоптимально. Применяют логарифмирование таргета $y_{new} = \ln(y+1)$, а при прогнозировании возвращают исходный масштаб через экспоненту. В sklearn для этого используется автоматизированная обертка `TransformedTargetRegressor`.

```

1 from sklearn.compose import TransformedTargetRegressor
2 from sklearn.linear_model import LinearRegression
3
4 # Модель автоматически применит log1p к y при fit() и expm1() при predict()
5 log_regressor = TransformedTargetRegressor(
6     regressor=LinearRegression(),
7     func=np.log1p,
8     inverse_func=np.expm1
9 )

```

Листинг 38: Автоматическое логарифмирование таргета

11.4 4. Сквозные Пайплайны (ColumnTransformer + Pipeline)

Для автоматизации предобработки разных типов данных используется связка ColumnTransformer (группировка по колонкам) и Pipeline (последовательные шаги обработки и финальная оценка).

```

1 from sklearn.compose import ColumnTransformer
2 from sklearn.pipeline import Pipeline
3
4 # 1. Выделяем списки колонок по типам обработки
5 numeric_features = ['sqft_living', 'bathrooms', 'sqft_lot']
6 categorical_features = ['waterfront', 'condition']
7 discretize_features = ['yr_built']
8
9 # 2. Создаем комплексный обработчик колонок
10 preprocessor = ColumnTransformer(
11     transformers=[
12         ('num', StandardScaler(), numeric_features),
13         ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), categorical_features),
14         ('bin', KBinsDiscretizer(n_bins=6, encode='onehot', strategy='uniform'),
15         discretize_features)
16     ],
17     remainder='drop' # Все остальные неуказанные колонки будут удалены
18 )
19
20 # 3. Объединяем препроцессор и модель в единый Pipeline
21 main_pipeline = Pipeline(steps=[
22     ('preprocessor', preprocessor),
23     ('model', LinearRegression())
24 ])
25
26 # Обучение всей цепочки: препроцессор делает fit_transform, модель делает fit
27 main_pipeline.fit(X_train, y_train)
28
29 # Предсказание: препроцессор делает transform, модель делает predict
30 predictions = main_pipeline.predict(X_test)

```

Листинг 39: Сборка комплексного пайплайна

11.5 5. Регуляризация (Ridge и Lasso)

Для предотвращения переобучения и уменьшения нормы весов в функцию потерь вводится регуляризационное слагаемое (штраф за сложность):

- **Ridge (L_2 -штраф):** $Q_{Ridge}(w) = MSE(w) + \alpha \sum_{j=1}^d w_j^2$. Ограничивает величину весов, плавно приближая их к нулю.
- **Lasso (L_1 -штраф):** $Q_{Lasso}(w) = MSE(w) + \alpha \sum_{j=1}^d |w_j|$. Из-за сингулярности геометрии ограничений зануляет коэффициенты у наименее информативных признаков, реализуя встроенный отбор признаков (*feature selection*).

11.6 6. Кросс-валидация и подбор параметров (GridSearchCV)

Поиск оптимальной силы регуляризации α проводится методом решетчатого поиска на кросс-валидации. При обращении к параметрам компонентов внутри Pipeline используется синтаксис с двойным подчеркиванием: `имякомпонента__имяпараметра`.

```

1 from sklearn.model_selection import GridSearchCV
2 from sklearn.linear_model import Ridge
3 from sklearn.metrics import mean_absolute_error
4

```

```

5 # Пересоздаем пайплайн с регуляризатором Ridge
6 ridge_pipeline = Pipeline([
7     ('preprocessor', preprocessor),
8     ('model', Ridge())
9 ])
10
11 # Задаем сетку параметров проверяем( параметры препроцессора и модели)
12 param_grid = {
13     'preprocessor__bin__n_bins': [4, 6, 8],
14     'model__alpha': [0.1, 1.0, 10.0, 100.0]
15 }
16
17 # Инициализируем поиск (5-fold кроссвалидация-, оптимизируем MSE)
18 grid_search = GridSearchCV(
19     estimator=ridge_pipeline,
20     param_grid=param_grid,
21     cv=5,
22     scoring='neg_mean_squared_error',
23     n_jobs=-1
24 )
25
26 # Запуск поиска оптимальной конфигурации
27 grid_search.fit(X_train, y_train)
28
29 # Вывод лучших параметров и оценка качества лучшей модели на тесте
30 print("Лучшие параметры:", grid_search.best_params_)
31 best_model = grid_search.best_estimator_
32 test_preds = best_model.predict(X_test)
33 print("Итоговый MAE на тесте:", mean_absolute_error(y_test, test_preds))

```

Листинг 40: Подбор гиперпараметров пайплайна через GridSearchCV

12 Градиентный спуск и методы оптимизации

12.1 Зачем реализовывать градиентный спуск вручную, если есть sklearn?

Готовые классы в `sklearn` (`LinearRegression`, `SGDRegressor`) жестко ограничены стандартными математическими функциями потерь: среднеквадратичной ошибкой (MSE), абсолютной ошибкой (MAE) или функцией Хубера.

В реальном бизнесе ошибки редко бывают симметричными. Например, в задаче прогнозирования спроса на авиабилеты для авиакомпании *недопрогноз* на 1 билет стоит очень дорого (упущенная прибыль от клиента составляет \$500), а *перепрогноз* обходится дешевле (билет остался нераспроданным, потеряно всего \$20). Стандартная MSE штрафует за обе ошибки одинаково. Написав цикл градиентного спуска вручную, можно подставить абсолютно любой асимметричный функционал ошибки, взять от него частные производные по параметрам w и обучить модель максимизировать реальную бизнес-метрику.

Также все современные алгоритмы глубокого обучения (`Adam`, `RMSprop`, `Adagrad`) являются итерационными надстройками над классическим градиентным спуском. Ручная реализация базового спуска позволяет легко добавлять в него новые механики:

- **Momentum (Инерция):** Добавление учета предыдущего направления движения, чтобы алгоритм быстрее проскакивал локальные минимумы и пологие плато.
- **Кастомные расписания (Learning Rate Decay):** Самостоятельное управление изменением шага по ходу итераций, например, уменьшение шага каждые N эпох или при замедлении падения ошибки.

12.2 1. Теоретический базис и концепция антиградиента

Обучение модели заключается в минимизации эмпирического риска: $Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \mathcal{L}(a(x_i), y_i) \rightarrow \min_w$.

- **Градиент $\nabla Q(w)$:** Вектор частных производных функции потерь по всем настраиваемым весам. Он указывает направление наискорейшего *роста* функции.
- **Антиградиент $-\nabla Q(w)$:** Направление наискорейшего *убывания* функции потерь. Вдоль него совершается шаг оптимизации.
- **Формула шага:** $w^{(t)} = w^{(t-1)} - \eta_t \nabla Q(w^{(t-1)})$, где η_t — темп обучения (*learning rate*).
- **Градиент для MSE в матричной форме** принимает вид: $\nabla_w Q(w) = \frac{2}{\ell} X^T (Xw - y)$.

12.3 2. Классический градиентный спуск (Full Gradient Descent)

Вычисляет градиент по абсолютно всей выборке на каждой итерации. Работает стабильно, траектория спуска гладкая, но метод вычислительно неэффективен на больших объемах данных.

```
1 import numpy as np
2
3 def full_gradient_descent(X, y, lr=0.01, num_iters=1000):
4     num_objects, num_features = X.shape
5     # Инициализация весов нулями
6     w = np.zeros(num_features)
7     loss_history = []
8
9     for i in range(num_iters):
10        # Вычисление предсказаний и ошибки
11        predictions = X.dot(w)
12        error = predictions - y
13
14        # Расчет аналитического градиента для MSE
15        gradient = (2.0 / num_objects) * X.T.dot(error)
16
17        # Обновление вектора параметров
18        w = w - lr * gradient
19
20        # Фиксация текущего значения функции потерь
21        current_loss = np.mean(error ** 2)
22        loss_history.append(current_loss)
23
24    return w, loss_history
```

Листинг 41: Кастомная реализация полного градиентного спуска

12.4 3. Стохастический градиентный спуск (Stochastic Gradient Descent, SGD)

Вместо подсчета градиента по всей матрице X , на каждом шаге случайным образом выбирается ровно один объект (x_i, y_i) , и шаг делается на основе его «шумного» градиента: $\nabla_w Q \approx \nabla_w \mathcal{L}(a(x_i), y_i)$.

- **Плюсы:** Скорость одной итерации составляет $O(d)$ вместо $O(\ell d)$; позволяет выходить из локальных минимумов за счет стохастической осцилляции.

- **Минусы:** Траектория спуска пилообразная («хаотичная»); модель не сходится к точному минимуму, а совершает случайные колебания вокруг него.

```

1 def stochastic_gradient_descent(X, y, lr=0.01, num_iters=5000):
2     num_objects, num_features = X.shape
3     w = np.zeros(num_features)
4     loss_history = []
5
6     for i in range(num_iters):
7         # Случайный выбор индекса одного объекта
8         random_index = np.random.randint(0, num_objects)
9         x_sample = X[random_index]
10        y_sample = y[random_index]
11
12        # Градиент на основе выбранного единичного объекта
13        prediction = np.dot(x_sample, w)
14        error = prediction - y_sample
15        gradient = 2.0 * x_sample * error
16
17        # Шаг в сторону антиградиента
18        w = w - lr * gradient
19
20        # Контрольный замер MSE по всей выборке для логов
21        full_loss = np.mean((X.dot(w) - y) ** 2)
22        loss_history.append(full_loss)
23
24    return w, loss_history

```

Листинг 42: Кастомная реализация стохастического градиентного спуска

12.5 4. Динамика и регулирование темпа обучения (Learning Rate)

Величина шага η — ключевой гиперпараметр. Неверный подбор разрушает сходимость.

- **Слишком большой η :** Алгоритм расходится, шаги перелетают через минимум, значения весов лавинообразно увеличиваются.
- **Слишком маленький η :** Модель обучается чрезмерно долго, застревает в пологих областях функции потерь.
- **Расписание уменьшения шага (*Learning Rate Decay*):** Для гарантированной сходимости SGD темп обучения обязан постепенно уменьшаться с течением времени. Классическое расписание: $\eta_t = \frac{\eta_0}{t^k}$ или $\eta_t = \eta_0 \cdot \gamma^t$.

12.6 5. Промышленный SGD-оптимизатор в sklearn: SGDRegressor

Для реальных задач используется класс `SGDRegressor`. Он поддерживает онлайн-обучение через метод `partial_fit`, а также включает встроенную регуляризацию L_1 и L_2 , рассчитываемую на каждом стохастическом шаге.

```

1 from sklearn.linear_model import SGDRegressor
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.model_selection import GridSearchCV
5
6 # Сборка пайплайна масштабирование( строго обязательно для градиентных методов!)
7 sgd_pipeline = Pipeline([
8     ('scaler', StandardScaler()),
9     ('model', SGDRegressor(random_state=42))
10 ])
11
12 # Сетка гиперпараметров для управления градиентным спуском и регуляризацией
13 # loss='squared_error' задает MSE оптимизацию
14 # penalty определяет тип штрафа (l1, l2, elasticnet)
15 # learning_rate='invscaling' уменьшает шаг по формуле: eta = eta0 / (t^power_t)
16 param_grid = {
17     'model__alpha': [0.0001, 0.001, 0.01],
18     'model__penalty': ['l2', 'l1'],
19     'model__learning_rate': ['invscaling', 'constant'],
20     'model__eta0': [0.01, 0.05, 0.1]
21 }
22
23 grid_search = GridSearchCV(
24     estimator=sgd_pipeline,
25     param_grid=param_grid,
26     cv=5,

```

```

27     scoring='neg_mean_squared_error',
28     n_jobs=-1
29 )
30
31 grid_search.fit(X_train, y_train)
32 print("Оптимальные параметры оптимизатора:", grid_search.best_params_)
33 best_sgd_model = grid_search.best_estimator_

```

Листинг 43: Применение SGDRegressor, сборка в Pipeline и подбор параметров

13 Предобработка данных и Feature Engineering (Полный курс)

13.1 1. Пропущенные значения (Missing Values)

В реальных задачах значения некоторых признаков у некоторых объектов отсутствуют. Это происходит по разным причинам: ошибки при записи данных, отказ респондента отвечать на вопрос, невозможность описать конкретное свойство у конкретного объекта (например, пропуски у автомобилей в графе «объем топливного бака»). Многие алгоритмы машинного обучения (в частности, линейная регрессия) не могут работать с пропущенными данными, поэтому эти пропуски необходимо заполнить.

Детальный анализ стратегий заполнения и их последствий:

1. **Константным уникальным значением (например, -999 или "unknown"):** Неудачный вариант для линейных методов (модель начнет считать пропуск близким к некоторому другому значению выборки), но быстрый и популярный способ со многими другими алгоритмами в машинном обучении (например, с решающими деревьями, которые могут выделить константу в отдельный лист).
2. **Средним арифметическим, медианой, модой:** Сохраняет базовые статистики выборки (например, математическое ожидание), но приводит к потере информации о наличии пропуска в данных. Искусственно занижает дисперсию признака и искажает форму распределения.
3. **Предсказаниями другого алгоритма:** Признак с пропусками временно выступает в роли целевой переменной, а остальные колонки используются как предикторы для вспомогательной модели. Способ точный, но затратный по времени (однако все равно не приносит принципиально новой информации в датасет, хотя и может положительно сказаться на метриках).
4. **Индикатор пропусков (Missing Indicator):** Важнейший инженерный подход, при котором создается дополнительный бинарный признак $I_{miss} \in \{0, 1\}$, принимающий значение 1, если исходное поле было пустым, и 0 — если оно было заполнено. Это позволяет линейной модели выучить отдельный вес для факта отсутствия данных и компенсировать смещение от заполнения средним.

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.impute import SimpleImputer
4
5 # Переменная add_indicator=True заставляет imputer параллельно создавать бинарные признаки пр
   опусков
6 imputer = SimpleImputer(strategy='median', add_indicator=True)
7
8 # Обучение трансформера ( fit ) происходит ИСКЛЮЧИТЕЛЬНО на обучающей выборке train
9 X_train_clean = imputer.fit_transform(X_train)
10 X_test_clean = imputer.transform(X_test)
```

Листинг 44: Комплексное заполнение пропусков с генерацией индикаторов

13.2 2. Категориальные признаки: ONE против Target Encoding

Категориальные признаки кодируют качественные свойства объектов и не имеют естественного арифметического порядка.

13.2.1 One-Hot Encoding (Унитарное кодирование)

Каждая уникальная категория преобразуется в отдельный бинарный столбец.

- **Дамми-ловушка (Dummy Variable Trap):** Линейная зависимость возникает, если сумма всех созданных ONE-столбцов константно равна 1 для каждого объекта. Это приводит к идеальной мультиколлинеарности со свободным коэффициентом w_0 (матрица $X^T X$ становится необратимой). В линейных моделях необходимо удалять один базовый столбец: параметр `drop='first'`.
- **Проблема высокого кардиналитета (High Cardinality):** Если уникальных значений у признака слишком много (например, ID города), ONE порождает сверхразреженную матрицу огромной размерности, что ведет к переобучению и взрыву памяти.

13.2.2 Target Encoding (Кодирование целевой переменной / Счетчики)

Каждая категория заменяется числом — средним значением целевой переменной по этой категории: $\hat{x}_i = \mathbb{E}[y \mid x = c_i]$.

Проблема утечки данных (Data Leakage) и жесткого переобучения: Если категория редкая или уникальная (например, ID пользователя), то среднее значение таргета по этой категории будет в точности равно таргету самого объекта. Модель мгновенно выучит идеальную зависимость на обучающей выборке, но полностью потеряет обобщающую способность на тестовых данных, так как произошла неявная утечка ответов в предикторы.

Математические методы регуляризации счетчиков:

1. **Сглаживание (Smoothing):** Добавление глобального ожидания целевой переменной μ_{global} по формуле:

$$\bar{x}_i = \frac{\sum y_{cat} + \alpha \cdot \mu_{global}}{N_{cat} + \alpha}$$

Где N_{cat} — число объектов данной категории, а α — вес сглаживания. Если категория редкая ($N_{cat} \rightarrow 0$), значение стремится к глобальной константе.

2. **Out-of-Fold (OOF) расчет (Кросс-фиттинг):** Выборка бьется на фолды. Значение счетчика для объекта из фолда A вычисляется по среднему значению таргета из оставшихся фолдов. Объект никогда «не видит» свой собственный таргет.

```
1 from sklearn.preprocessing import OneHotEncoder
2 from category_encoders import TargetEncoder
3
4 # handle_unknown='ignore' защищает от падения, если на тесте придет новая категория
5 ohe = OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)
6 X_train_ohe = ohe.fit_transform(X_train[['city']])
7
8 # TargetEncoder из category_encoders автоматически реализует OOF и smoothing под капотом
9 te = TargetEncoder(smoothing=20.0)
10 X_train_te = te.fit_transform(X_train[['user_id']], y_train)
```

Листинг 45: Безопасное кодирование признаков

13.3 3. Текстовые признаки: Частотные матрицы и TF-IDF

13.3.1 Bag of Words (Мешок слов) и n-граммы

- **Bag of Words:** Строится вектор длиной во весь уникальный словарь корпуса. Координата — число вхождений слова в документ. Порядок слов и синтаксис полностью игнорируются.
- **n-граммы:** Извлечение цепочек из n смежных слов (или символов). Пример для биграмм ($n = 2$): фраза «строгое проверяет работы» порождает токены «строгое проверяет», «проверяет работы». Позволяет ловить контекст и отрицания («не плохой»), но размер словаря растет экспоненциально с ростом n .

13.3.2 TF-IDF векторизация

Обычная частотность слов выделяет паразитные стоп-слова (союзы, предлоги). TF-IDF решает это, штрафую общепупотребительные токены:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

Где $\text{TF}(t, d) = \frac{n_{td}}{\sum_k n_{kd}}$ — частота токена t в документе d . Компонента $\text{IDF}(t, D) = \log \frac{|D|}{1 + |\{d \in D: t \in d\}|}$ — инверсная частота токена во всем корпусе документов D . Если слово встречается везде, его IDF стремится к 0.

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 # max_df=0.95 отсекает слова, встречающиеся более чем в 95 процентах документов корпуса
4 tfidf = TfidfVectorizer(ngram_range=(1, 3), max_features=10000, max_df=0.95)
5 X_train_text = tfidf.fit_transform(X_train['raw_text'])
```

Листинг 46: Построение частотных текстовых матриц

13.4 4. Лемматизация и стемминг

Одно и то же слово во флексиях (склонениях, спряжениях) интерпретируется частотными методами как абсолютно разные слова, что делает признаковое описание избыточным. Устранить эту проблему можно при помощи лемматизации и стемминга.

- **Стемминг:** Процесс нахождения основы слова. В результате однокоренные слова преобразуются к одинаковому виду. Работает по жестким эвристическим правилам отсечения окончаний. Например, «вагон» → «вагон», «вагонов» → «вагон», но при этом «важная» → «важн», «важно» → «важн». Стеммер просто «режет» слово, не проводя глубокий языковой анализ. Алгоритм: *Snowball Stemmer*.
- **Лемматизация:** Процесс приведения слова к его нормальной форме — лемме. Это процесс более сложный по сравнению со стеммингом. Требуется разбор части речи и контекста.

Правила приведения к лемме:

- Для существительных — именительный падеж, единственное число;
- Для прилагательных — именительный падеж, единственное число, мужской род;
- Для глаголов, причастий, деепричастий — глагол в инфинитиве (начальной форме).

```

1 from nltk.stem import SnowballStemmer
2 import pymorphy2
3
4 # Инициализация стеммера для русского языка
5 stemmer = SnowballStemmer('russian')
6 print(stemmer.stem('вагонами')) # Результат: вагон
7
8 # Инициализация морфологического анализатора лемм
9 morph = pymorphy2.MorphAnalyzer()
10 def clean_and_lemmatize(text):
11     words = text.split()
12     # parse(word)[0].normal_form возвращает строгую словарную лемму слова
13     return " ".join([morph.parse(w)[0].normal_form for w in words])
14
15 print(clean_and_lemmatize('вагонами грузили')) # Результат: вагон грузить

```

Листинг 47: Применение SnowballStemmer и pymorphy2

14 Предсказание и калибровка вероятностей

14.1 Прогнозирование вероятностей (Мягкая классификация)

Задача мягкой классификации требует от алгоритма $\hat{p}(x)$ аппроксимации истинной условной вероятности класса $p(y = +1|x)$. Качество мягкой классификации нельзя измерять только метриками жесткой классификации (accuracy, precision, recall) или ранжирования (ROC-AUC).

Минимум матожидания корректной функции потерь должен достигаться на истинной вероятности:

$$\arg \min_{\hat{p} \in [0,1]} [L(y, \hat{p})x] = p(y = +1|x)$$

Квадратичная функция потерь (Brier Score): $L(y, \hat{p}) = ([y = +1] - \hat{p})^2$, где $y \in \{0, 1\}$.

- **Математическое обоснование:** $[L(y, \hat{p})x] = p(y = 1|x)(\hat{p} - 1)^2 + (1 - p(y = 1|x))\hat{p}^2$.
- Дифференцирование по $\hat{p}$ дает: $2\hat{p} - 2p(y = 1|x) = 0 \implies \hat{p} = p(y = +1|x)$.
- **Применение:** Позволяет модели предсказывать корректно откалиброванные вероятности.

Абсолютная функция потерь: $L(y, \hat{p}) = |[y = 1] - \hat{p}|$, где $\hat{p} \in [0, 1]$.

- **Математическое обоснование:** $[L(y, \hat{p})x] = p(y = 1|x)(1 - \hat{p}) + (1 - p(y = 1|x))\hat{p}$.
- При $p \neq \frac{1}{2}$ критических точек внутри $(0, 1)$ нет, минимум матожидания всегда достигается на одном из концов отрезка: $\arg \min \in \{0, 1\}$.
- **Применение:** Не позволяет предсказывать вероятности, так как классификатор выдает только крайние жесткие значения (0 или 1).

14.2 Метрики качества вероятностей

- **Log Loss:** $\text{LogLoss} = -\frac{1}{n} \sum_{i=1}^n (y_i \log p_i + (1 - y_i) \log(1 - p_i))$. Идеал = 0, худший случай $\rightarrow \infty$.
- **Brier Score:** $\text{BrierScore} = \frac{1}{n} \sum_i (y_i - p_i)^2$. Идеал $\rightarrow 0$, худший случай = 1.
- **Калибровочная кривая:** Абсцисса — предсказанная вероятность, ордината — доля положительных примеров. В идеальном случае совпадает с прямой $y = x$.

14.3 Методы калибровки вероятностей

Для получения откалиброванной модели строится надстройка над базовым алгоритмом $f(x)$, переводящая его сырые выходы в вероятности, близкие к эмпирическим.

Калибровка Платта (Platt Calibration):

$$P(y = 1|x) = \frac{1}{1 + \exp(af(x) + b)}$$

- **Суть:** Применение логистической регрессии поверх предсказаний базового алгоритма без регуляризации. Параметры a, b настраиваются методом максимума правдоподобия на отложенной выборке.
- **Сглаживание меток (против переобучения):** $t_+ = \frac{N_+ + 1}{N_+ + 2}$ для положительных и $t_- = \frac{1}{N_- + 2}$ для отрицательных примеров.
- **Применяется:** На малых выборках или когда искажение базовой модели имеет сигмоидальный вид (например, SVM).
- **Не применяется:** Если форма искажения вероятностей сложная и нелинейная.

Изотоническая регрессия (Isotonic Regression):

$$P(y = 1|x) = m(f(x)), \quad \text{где } m(t) \text{ — кусочно-постоянная неубывающая функция}$$

- **Суть:** Настройка под квадратичную ошибку: $m = \arg \min_z \sum (y_i - z(f(x_i)))^2$ с помощью алгоритма PAV (Pool-Adjacent-Violators).
- **Применяется:** Когда обучающая выборка велика и форма искажения кривой заранее неизвестна.
- **Не применяется:** На малых выборках из-за высокого риска сильного переобучения.

Beta Calibration: Развитие калибровки Платта с тремя параметрами:

$$P(y = 1|x) = \left(1 + \frac{1}{\exp(c) \cdot \frac{f(x)^a}{(1-f(x))^b}} \right)^{-1}$$

- **Суть:** Добавление третьего параметра обеспечивает аппроксимацию более широкого класса кривых.
- **Свойство:** Семейство включает прямую $y = x$, поэтому калибровка не ухудшит уже откалиброванную базовую модель.

14.4 Мультиклассовая калибровка

Каждый класс калибруется отдельно, после чего векторы вероятностей нормируются ($\sum_i p_i = 1$).

- **One-vs-All:** Обучается n классификаторов. Многоклассовая вероятность: $\tilde{p}_i = \frac{\hat{p}_i}{\sum_j \hat{p}_j}$, где $\hat{p}_i$ — откалиброванный бинарный выход задачи « i против всех».
- **One-vs-One:** Обучается $\frac{n(n-1)}{2}$ парных классификаторов. Сумма вероятностей в парах: $\tilde{p}_i = \frac{1}{K-1} \sum_{j \neq i} p_{ij}$, после чего значения нормируются: $\hat{p}_i = \frac{\tilde{p}_i}{\sum_{k=1}^K \tilde{p}_k}$.

15 Квантильная регрессия

15.1 Постановка задачи и функционал ошибки

Применяется в задачах, где издержки занижения и завышения прогнозов асимметричны (например, прогноз спроса товаров интернет-магазина на складе).

$$Q(a, X^\ell) = \sum_{i=1}^{\ell} \rho_{\tau}(y_i - a(x_i))$$

Квантильная функция потерь (Pinball Loss):

$$\rho_{\tau}(z) = (\tau - 1)[z < 0]z + \tau[z \geq 0]z = (\tau - \frac{1}{2})z + \frac{1}{2}|z|$$

Параметр $\tau \in [0, 1]$ определяет соотношение важности знака ошибки. Чем больше τ , тем выше штраф за занижение прогноза.

15.2 Вероятностный смысл функционала

Пусть в каждой точке $x \in$ задано вероятностное распределение $p(yx)$ на возможных ответах. Оптимальное число q минимизирует функционал:

$$Q = \int_p \tau(y - q)p(yx) dy$$

Продифференцируем выражение по параметру q (используя правила дифференцирования интегралов, зависящих от параметра):

$$\frac{\partial Q}{\partial q} = (1 - \tau) \int_{-\infty}^q p(yx) dy - \tau \int_q^{\infty} p(yx) dy = 0$$

Отсюда получаем следующее соотношение:

$$\frac{\tau}{1 - \tau} = \frac{\int_{-\infty}^q p(yx) dy}{\int_q^{\infty} p(yx) dy}$$

Данное уравнение справедливо тогда и только тогда, когда q является τ -квантилью распределения $p(yx)$. Таким образом, оптимизация функции потерь $\rho_{\tau}(z)$ приводит к тому, что алгоритм $a(x)$ приближает условную τ -квантиль ответов в каждой точке.

15.3 Сводная таблица аппроксимации распределений

Функция потерь	Математический функционал	Что приближает алгоритм $a(x)$
Квадратичная (MSE)	$L(y, a) = (y - a)^2$	Условное матожидание $[yx]$
Абсолютная (MAE)	$L(y, a) = y - a $	Медиану распределения $\text{median}[p(yx)]$
Квантильная (ρ_{τ})	$L(y, a) = \rho_{\tau}(y - a)$	τ -квантиль распределения $p(yx)$

Таблица 1: Вероятностный смысл оптимизационных функционалов в регрессии.